FUZZCODER: Byte-level Fuzzing Test via Large Language Model

Liqun Yang[1], **Jian Yang**[1,*], **Chaoren Wei**[1], **Guanglin Niu**[2], **Ge Zhang**[3,5], **Yunli Wang**[1], **Linzheng Chai**[1], **Wanxu Xia**[1], **Hongcheng Guo**[1], **Shun Zhang**[1], **Jiaheng Liu**[1], **Yuwei Yin**[1], **Junran Peng**[4], **Jiaxin Ma**[6], **Liang Sun**[1] **Zhoujun Li**[1]
[1]Beihang University; [2]University of British Columbia; [3]University of Waterloo
[4]University of Science and Technology Beijing; [5]M-A-P;
[6]Beijing University of Posts and Telecommunications
weichaoren@buaa.edu.cn;

arXiv:2409.01944v1 [cs.CL] 3 Sep 2024

Abstract

Fuzzing is an important dynamic program analysis technique designed for finding vulnerabilities in complex software. Fuzzing involves presenting a target program with crafted malicious input to cause crashes, buffer overflows, memory errors, and exceptions. Crafting malicious inputs in an efficient manner is a difficult open problem and the best approaches often apply uniform random mutations to pre-existing valid inputs. In this work, we propose to adopt fine-tuned large language models (FUZZCODER) to learn patterns in the input files from successful attacks to guide future fuzzing explorations. Specifically, we develop a framework to leverage the code LLMs to guide the mutation process of inputs in fuzzing. The mutation process is formulated as the sequence-to-sequence modeling, where LLM receives a sequence of bytes and then outputs the mutated byte sequence. FUZZCODER is fine-tuned on the created instruction dataset (Fuzz-Instruct), where the successful fuzzing history is collected from the heuristic fuzzing tool. FUZZCODER can predict mutation locations and strategies locations in input files to trigger abnormal behaviors of the program. Experimental results show that FUZZCODER based on AFL (American Fuzzy Lop) gain significant improvements in terms of effective proportion of mutation (EPM) and number of crashes (NC) for various input formats including ELF, JPG, MP3, and XML.[1]

1 Introduction

Fuzzing test (Guo et al., 2018; Xie et al., 2022; Wei et al., 2022; Cummins et al., 2018; Manès et al., 2019; Li et al., 2018), a dynamic software testing technique, has emerged as a powerful method for uncovering vulnerabilities and defects within software applications. Fuzzing frameworks like AFL (American Fuzzy Lop) and libFuzzer have

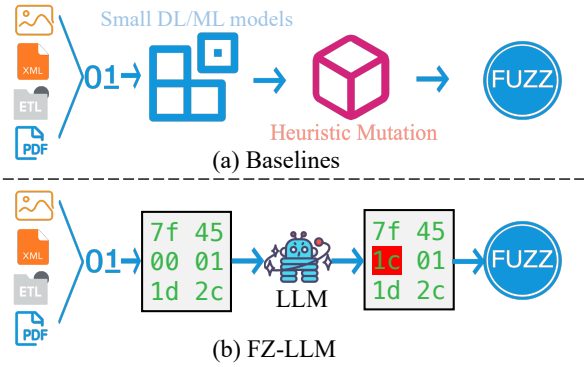

Figure 1: Comparison between the standard byte-level fuzz test and our proposed method.

become industry standards, while researchers further explore advanced strategies like evolutionary fuzzing and hybrid approaches to enhance test case generation and code coverage. As the intricacy of software systems escalates, fuzzing continues to evolve, proving its essential role in the realm of software development and security testing.

Based on neural network architectures like RNNs and GANs (Goodfellow et al., 2016), this line of research has shown potential in improving test case generation, increasing code coverage, and detecting elusive vulnerabilities. Trained on billions of lines of code, large language models (LLMs) have shown exceptional aptitude in various software engineering tasks in code generation (Rozière et al., 2023; Bai et al., 2023; Guo et al., 2024a), program repair (Zhang et al., 2023; Guo et al., 2023), and fuzzing (Xia et al., 2024; Deng et al., 2023; Huang et al., 2024; Yang et al., 2024). The rigorous pre-training on vast code datasets forms the cornerstone of the capabilities of LLM in code generation and comprehension, even for the encoded byte sequence. Byte level byte pair encoding (BBPE) tokenizer (Wang et al., 2020; Wu et al., 2024; Radford et al., 2019) have become the standard practices for state-of-the-art LLMs, which brings powerful understanding and genera-

*Corresponding author.
[1]https://github.com/weimo3221/FUZZ-CODER

tion capability for byte-like data. Moreover, these LLMs can be further optimized through fine-tuning or prompting to enhance their proficiency in specific domains. *However, how to effectively leverage instruction fine-tuning (IFT) to inspire LLMs to help byte-based mutation for the fuzzing test still requires further exploration.*

In this paper, we investigate the feasibility of leveraging code LLM to develop a framework, guiding the mutation process of inputs in fuzzing. The mutation process is formulated as the sequence-to-sequence modeling, where LLM receives a byte sequence and then outputs the mutated byte sequence. The LLM is fine-tuned on the created instruction dataset, where the successful fuzzing history is collected from the heuristic fuzzing tool. In Figure 1, the instruction corpus is coupled into pairs comprised of original inputs and successfully mutated inputs. FUZZCODER aims at identifying the most possible bytes within input files for mutations. To gather the instruction dataset Fuzz-Instruct, we initially adopt standard fuzzing methods to record mutation instances that yield new code coverage or trigger crashes. Fuzz-Instruct then serves to train FUZZCODER based on different code foundation models to guide towards generating promising mutated inputs. While our methodology is adaptable to various fuzzing frameworks, we apply it specifically to the state-of-the-art AFL, which introduces random mutations into a batch of seed input files and curates a queue of new inputs, which are effective in tracing new code executions.

Our proposed method is evaluated on the benchmark Fuzz-Bench, comprised of 8 programs: `NM_ELF`, `READ_ELF`, `OBJDUMP_ELF`, `LINT_XML`, `MP3GAIN_MP3`, `IMAGEMAGICK_GIF`, `SPLIT_TIFF`, and `TRAN_JPEG`. Fuzz-Bench accepts the different format inputs, including ELF, XML, MP3, and GIF. FUZZCODER significantly improves line coverage and branch coverage compared to the previous strong baselines. Further, we observe that FUZZCODER triggers more new paths or the frequency of code blocks found during fuzz testing due to the effective mutation prediction of the understanding capability of the code LLM.

The key contributions are summarized as:

- We formulate the fuzzing test as a sequence-to-sequence paradigm and then introduce the generation model to attack vulnerable positions by selecting proper mutation positions and strategies. The data in any format is first converted into a sequence of bytes as the input of LLMs. Then, the code LLM will decide the possible mutation strategies and positions.
- We construct a complete framework to fine-tune the code LLMs with the help of the collected instruction corpora Fuzz-Instruct. To effectively evaluate the performance of different models, we construct a fuzzing test benchmark Fuzz-Bench comprised of 8 programs, which accept different formats of data (e.g. ELF, JPG, MP3, and XML).
- The experimental results on created benchmark Fuzz-Bench (simulation using AFL) demonstrate the fine-tuned FUZZCODER significantly improves the effective proportion of mutation (EPM) and triggers more program crashes compared to the previous baselines.

2 Preliminary: Fuzzing Test

Fuzzing is a robust software testing technique designed to uncover vulnerabilities and flaws in computer programs, primarily by subjecting them to a barrage of unexpected and often invalid inputs. The fuzzing test can be mathematically represented as follows:

$$\mathcal{F}(T, g(x)) = R \tag{1}$$

where $\mathcal{F}(\cdot, \cdot)$ represents the fuzzing process receiving mutation of input test cases. T is the target software or program subjected to the fuzzing test. I represents the input test cases, which are typically malformed, unexpected, or random data. $g(x)$ is the mutation format of the original input x. R stands for the results or observations obtained during the fuzzing test, which may include system crashes, error messages, or other unexpected behaviors in the target software.

American Fuzzy Lop[2] (AFL) is a widely used automated vulnerability mining tool, which finds security vulnerabilities in software programs through fuzzy testing techniques. Fuzzy testing is a black-box testing methodology that injects random or semi-random data into program inputs to detect anomalous behavior and potential vulnerabilities in the program. In AFL, **mutation** refers to the generation of new fuzzy test inputs by modifying the input samples, which is a core component of

[2] https://github.com/google/AFL

AFL fuzzy testing. Its mutation strategy employs a range of random and semi-randomized mutation techniques to create a diversity of test inputs. Let $x^{(i)} \in \{x^{(1)}, \ldots, x^{(n)}\}$ denote the seed test input from the initial pool comprised of n test cases, we leverage the NLP techniques to generate the mutated test case $z^{(i)}$. Different from the rule-based mutation, we use a generation model to obtain variant samples for fuzzy testing by predicting variant locations and variant types. Specifically, $x^{(i)} = \{x_1^{(i)}, \ldots, x_m^{(i)}\}$ is m bytes input sequence, the prediction model $\mathcal{M}$ chooses k mutation positions $p = \{p_1, \ldots, p_k\}$ and their corresponding mutation strategies $s = \{s_1, \ldots, s_k\}$ to modify the original test case x^k into z^k. The process can be described as:

$$P(p, s|x^{(i)}) = \prod_{j=1}^{m} P(p_j, s_j|x^{(i)}, p_{<j}, s_{<j}; \Theta) \quad (2)$$

where $p_{<j} = (p_1, \ldots, p_{j-1})$ and $s_{<j} = (s_1, \ldots, s_{j-1})$. p_j and s_j represent the j-th mutation position and mutation strategy respectively predicted by the previous context $p_{<j}$ and $s_{<j}$ sequentially and the original test case $x^{(i)}$.

3 Fuzz-Bench

We introduce 8 fuzzing datasets: `NM_ELF`, `READ_ELF`, `OBJDUMP_ELF`, `LINT_XML`, `MP3GAIN_MP3`, `IMAGEMAGICK_GIF`, `SPLIT_TIFF`, and `TRAN_JPEG`, which accept the different format inputs, including the ELF, XML, MP3, and GIF format. The program subjected to the fuzzing test originates from the FuzzBench[3] and previous works[4].

Here, we describe the details of each dataset. For `LINT_XML`, the program parses one or more XML files and prints various types of output, depending upon the options selected. It is useful for detecting errors both in XML code and in the XML parser itself. For `READ_ELF`, the program reads and displays information about the contents of ELF (executable and linkable Format) format files, which include executables, target files, and shared libraries. For `NM_ELF`, the program displays symbol table information in target files (including executables, target files, and shared libraries). The symbol table contains symbols defined and referenced in the program (e.g., variable names, function names, etc.) and their associated attributes. For `OBJDUMP_ELF`, the program displays various information from object files (including executable files, target files, and shared libraries), such as disassembled code and section table information. For `MP3GAIN_MP3`, the program adjusts the volume of MP3 audio files, which aims to balance and normalize the volume of MP3 files so that they sound more consistent when played without noticeable volume differences. For `IMAGEMAGICK_GIF`, the program is a tool in ImageMagick for processing various image files (including JPG, PNG, GIF, etc.). It can get information about the image, adjust the image, and process it. For `SPLIT_TIFF`, it splits a TIFF file containing multiple images into multiple separate TIFF files, each file containing a frame or page from the input file. For `TRAN_JPEG`, it can rotate JPG images 90 degrees, 180 degrees or 270 degrees clockwise. JPG images can also be cropped, optimized, etc.

Data Construction For different programs, we need to collect the data used for LLMs separately by fuzzing the programs with heuristic methods, where the baseline is denoted as AFL. Through the simulation of the original AFL, we can collect the k valid mutations $\{(p_1, s_1), \ldots, (p_k, s_k)\}$ for the specific test case x. Then, we can construct the supervised training pair (x, p, s) comprised of the input test case x, valid mutation positions p, and the corresponding strategies s. For each dataset, we can obtain the corresponding instruction corpus $D_t = \{I^{(i)}, x^{(i)}, y^{(i)}\}_{i=1}^{N_t}$ ($1 \leq t \leq T = 8$, T is the number of the programs, N_t is the training data size of the program t, and $I^{(i)}$ is the instruction) and merge them as the whole dataset $D = \{D_t\}_{t=1}^{T}$.

Given the specific test case, there exist different valid mutation strategies to successfully fuzz the program (e.g. the mutation leads to the program crash or triggers a new execution path). We can gather the valid mutation pairs together as the target sequence. i.e., valid (p_i, s_i) pairs of the test case. In the following example, if its valid (p_i, s_i) pairs are $(1, 2)$ and $(1, 3)$, it denotes that the 2-th and 3-th token in the hexadecimal sequence will perform 1-th operation to cause crash of the program. the final expression can be described as follows:

Data Collection

Byte Input: 0x3c 0x21 0x44 0x4f 0x43
Mutation strategies: $[(1, 2), (1, 3)]$

[3] https://github.com/google/FuzzBench
[4] https://github.com/fdu-sec/NestFuzz

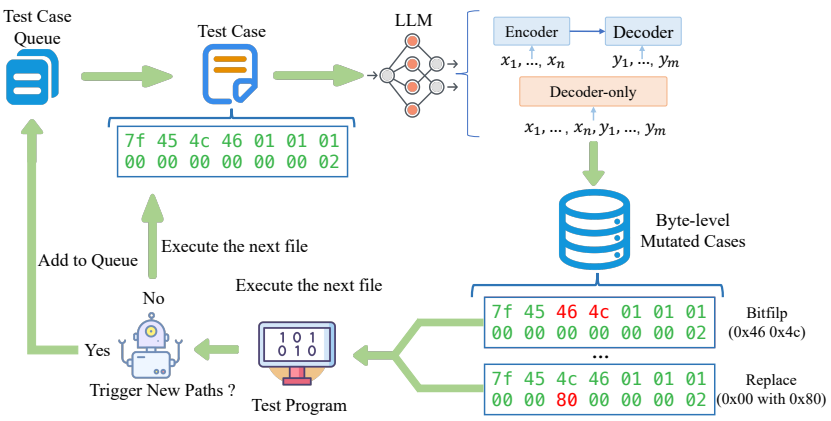

Figure 2: The workflow of the fuzzing test with fine-tuned LLMs FUZZCODER.

The queue of input sequences Q is used to store input test cases (test cases). When the fuzzing process (e.g. AFL) starts, it automatically selects and mutates input data based on the response of the target program to better explore potential program paths and boundary conditions. Q contains input files that successfully caused the program to execute different paths during testing. These input files are considered valid because they cause program execution to enter new code paths or trigger specific error conditions. To collect as much mutation data as possible for each program, each program is fuzzed multiple times.

Data Split Since the training of the model requires a training set and a valid set, we randomly select 90% of the samples as the training set and 10% of the data as the valid set. The number of samples is described as:

Benchmark	Train	Test	Program	Input	Option
NM_ELF	4534	504	*nm-new*	ELF	-a @@
READ_ELF	4167	464	*readelf*	ELF	-a @@
OBJDUMP_ELF	4009	446	*objdump*	ELF	-x -a -d @@
LINT_XML	5442	605	*xmllint*	XML	–valid –recover @@
MP3GAIN_MP3	1431	150	*mp3gain*	MP3	@@
IMAGEMAGICK_GIT	6477	720	*magick*	GIF	identify @@
SPLIT_TIFF	4136	459	*tiffsplit*	TIFF	@@
TRAN_JPEG	1376	153	*jpegtran*	JPEG	@@

Table 1: Statistics of the different benchmarks.

Simulation Environment We incorporate the generation model into the AFL framework to support the fuzzing with LLM. The simulation environment is Ubuntu 18.04.6 LTS, Intel Xeon Processor (Skylake, IBRS), A100-PCIE-40GB, AFL-2.57b[5].

[5] https://github.com/google/AFL

4 Fuzzing Test via Generation Model

4.1 Input Encoding

Our framework consists of a fuzzer and a model that highlights useful locations in an input file. During runtime. the fuzzer queries the model for each seed file and focuses mutations on the highlighted locations. Given an open-ended input file, we first convert the input file into a sequence of bytes $x^{(i)}$ in Figure 2 (hexadecimal sequence). Then, the generation model should predict the mutation positions $p = \{p_1, \ldots, p_k\}$ and the mutation strategies $s = \{s_1, \ldots, s_k\}$, where the s_k is the corresponding mutation strategy of the position p_k. To jointly model the mutation position and strategy, the prediction sequence $y = (y_1, \ldots, y_{2k})$ can be described as:

$$y = (p_1, s_1, \ldots, p_k, s_k) \tag{3}$$

where the model first predicts the mutation position p_k and then output the corresponding strategy s_k.

4.2 Encoder-Decoder Framework

Given the source inputs D_{src} and target predictions D_{trg}, the encoder of the encoder-decoder-based FUZZCODER first receives the original input x and encodes it into the hidden states H_{enc} with the bidirectional attention mechanism.

$$H_e = \mathcal{S}(x, \mathcal{M}_e) = \mathop{\Big\|}_{a=1}^{A} \texttt{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} \otimes \mathcal{M}_e\right) V \tag{4}$$

where A is the number of attention heads Then, the decoder predicts the target tokens sequentially based on H_e.

4.3 Decoder-only Framework

Given the source inputs D_{src} and target predictions D_{trg}, the encoder of the encoder-decoder-based FUZZCODER first receives the original input x and encodes it into the hidden states H_{enc} with the bidirectional attention mechanism.

$$H_d = \mathcal{S}(x, \mathcal{M}_d) = \mathop{\Big\|}_{a=1}^{A} \texttt{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} \otimes \mathcal{M}_d\right) V \tag{5}$$

where A is the number of attention heads The decoder predicts the target tokens sequentially based on H_e with the casual mask $\mathcal{M}_d$.

4.4 Mutation Strategy Prediction

For each mutation position p_j, we use the generation model to infer the possible mutation strategy for the position. 12 candidate mutation strategies are provided for each position, including: (1) bitflip 1/1: perform bitfilp on a bit randomly. (2) bitflip 2/1: perform bitfilp on two neighboring bits randomly. (3) bitflip 4/1: perform bitfilp on four neighboring bits randomly. (4) bitflip 8/8: randomly select a byte and XOR it with 0xff. (5) bitflip 16/8: randomly select two neighboring bytes and XOR them with 0xff. (6) bitflip 32/8: randomly select four neighboring bytes and XOR them with 0xff. (7) arith 8/8: randomly select a byte and perform addition or subtraction on it (operands are 0x01 0x23). (8) arith 16/8: randomly select two neighboring bytes and convert these two bytes into a decimal number. Select whether to swap the positions of these two bytes. Perform addition or subtraction on it (operands are 1 35). Finally, convert this number to 2 bytes and put it back to its original position. (9) arith 32/8: randomly select four neighboring bytes. Select whether to swap the positions of these four bytes. Convert these four bytes into a decimal number. Perform addition or subtraction on it. Finally, convert this number to 4 bytes and put it back to its original position. (10) interest 8/8: randomly select a byte and replace it with a random byte. (11) interest 16/8: randomly select two neighboring bytes and replace them with two random bytes. (12) interest 32/8: randomly select four neighboring bytes and replace them with four random bytes.

4.5 Jointly Training

Since the mutation strategies and positions $y = (p_1, s_1, \ldots, p_k, s_k)$ are our prediction goals, the supervised fine-tuning objective of FUZZCODER can be described as:

$$\mathcal{L}_m = -\mathbb{E}_{x^{(i)}, p^{(i)}, s^{(i)} \in D_{src}} \log P(p^{(i)}, s^{(i)} | x^{(i)}) \tag{6}$$

where $x^{(i)}$ is the i-th original input from the collected dataset. $p = (p_1, \ldots, p_k)$ is the predicted mutation positions and $s = (s_1, \ldots, s_k)$ is the mutation strategies.

Task Description:
Now, you are a AFL (American Fuzzy Lop), which is a highly efficient and widely used fuzz testing tool designed for finding security vulnerabilities and bugs in software. You are now fuzzing a program named {dataset_name}, which requires variable (a byte sequence) to run. I will give you a byte sequence as input sequence, and you need to mutate the input sequence to give me a output sequence through a mutation operation below. Finally you need to give me a output which includes input sequence, mutation operation and output sequence.

Mutation Operations:
{Mutation Operations O}

Input Sequence Definition:
It consists of bytes represented in hexadecimal, separated by spaces. It is the byte sequence to be mutated. It is a variable that can cause the program to crash or trigger a new path.

Output Sequence Definition:
It consists of bytes represented in hexadecimal, separated by spaces. It is the mutated byte sequence. It is a variable that can cause the program to crash or trigger a new path.

Input Sequence:
{byte_input}
Please list all possible mutation strategies (mutation position and mutation operation) with the JSON format as:
output:
{
"mutation strategies": [
(o_1, p_1),
... ,
(o_N, p_N),
]
}

Figure 3: The prompt to get mutation positions and strategies of FUZZCODER.

4.6 Incorporating LLMs into Fuzzing Test

The AFL tool will first compile our test program and then use the test cases after mutation as input into the compiled program. The mutated test case causing a crash or triggering a new path will be used as seeds. FUZZCODER adopts the Top-p sampling strategy to produce the candidate mutation strategy and position for diversity, which ensures that the effective mutation strategy and mutation positions are covered as much as possible.

5 Experiments

We evaluate our proposed method FUZZCODER on 8 test sets, including NM_ELF, READ_ELF,

Method	Base	Size	bitflip 1/1	bitflip 2/1	bitflip 4/1	bitflip 8/8	bitflip 16/8	bitflip 32/8	arith 8/8	arith 16/8	arith 32/8	interest 8/8	interest 16/8	interest 32/8	Avg.
							READ_ELF								
AFL (Original)	-	-	1.50	0.66	0.25	0.33	0.09	0.24	0.30	0.00	0.00	0.48	0.06	0.03	0.33
AFL (LSTM)	-	-	1.37	1.11	0.97	0.00	0.00	0.00	2.49	0.00	0.00	0.00	0.00	0.49	0.54
AFL (Transformer)	-	-	1.11	1.04	1.02	1.61	0.00	0.90	3.99	0.22	0.30	2.34	1.98	0.82	1.28
FuzzCoder	StarCoder-2	7B	3.42	0.92	1.28	2.45	0.12	0.15	0.63	0.12	0.05	0.45	2.41	0.34	1.03
FuzzCoder	StarCoder-2	15B	4.21	2.38	1.43	2.95	0.24	0.21	1.25	0.45	0.38	0.57	1.38	0.45	1.32
FuzzCoder	CodeLlama	7B	3.82	2.24	1.45	2.01	0.17	0.33	1.36	0.19	0.43	1.24	0.95	0.91	1.26
FuzzCoder	DeepSeek-Coder	7B	1.98	1.73	0.66	3.13	0.08	0.24	2.92	0.22	0.25	1.48	1.82	2.05	1.38
FuzzCoder	CodeQwen	7B	3.00	1.41	2.07	1.09	0.66	0.97	5.86	0.37	0.37	0.73	0.54	1.15	**1.52**
FuzzCoder	CodeShell	7B	2.08	2.42	1.34	3.81	0.54	0.55	2.45	0.55	0.02	0.45	0.25	1.23	1.31
							OBJ_DUMP								
AFL (Original)	-	-	2.07	0.89	0.43	0.43	1.35	1.93	0.31	0.08	0.01	0.79	0.21	0.11	0.72
AFL (LSTM)	-	-	1.26	4.20	2.95	1.21	1.23	2.81	1.33	1.67	0.00	2.78	2.45	2.64	2.04
AFL (Transformer)	-	-	1.97	1.68	0.86	0.00	1.38	1.84	1.27	1.61	1.47	1.82	1.01	1.28	1.35
FuzzCoder	StarCoder-2	7B	1.24	1.71	0.02	1.21	0.23	0.05	1.52	0.85	0.32	0.01	0.23	0.43	0.65
FuzzCoder	StarCoder-2	15B	1.37	1.74	0.11	2.48	0.08	0.73	1.78	0.43	0.55	0.07	0.11	1.28	0.89
FuzzCoder	CodeLlama	7B	1.62	1.32	0.18	1.15	0.49	2.43	0.75	0.19	0.37	0.05	0.14	1.15	0.82
FuzzCoder	DeepSeek-Coder	7B	1.74	1.10	0.50	2.00	1.21	3.45	6.84	1.70	3.45	1.44	1.63	1.47	**2.21**
FuzzCoder	CodeQwen	7B	1.16	0.95	0.46	6.23	1.05	0.82	3.87	0.36	1.27	0.42	1.08	1.44	1.59
FuzzCoder	CodeShell	7B	1.12	0.32	0.07	2.43	2.45	0.35	1.34	0.23	0.05	0.34	0.13	0.92	0.81
							NM								
AFL (Original)	-	-	1.35	0.41	0.04	0.38	2.03	1.29	0.10	0.01	0.00	0.23	0.03	0.05	0.49
AFL (LSTM)	-	-	1.95	0.84	0.09	9.74	0.00	0.90	2.47	0.00	0.00	0.24	0.75	0.72	1.47
AFL (Transformer)	-	-	0.90	0.83	0.30	3.48	1.27	1.31	3.80	1.32	0.00	0.00	1.29	0.52	1.25
FuzzCoder	StarCoder-2	7B	1.34	0.23	0.75	0.18	0.85	0.38	1.78	0.01	0.34	0.05	0.11	0.01	0.50
FuzzCoder	StarCoder-2	15B	1.41	0.37	1.21	0.34	0.93	0.72	2.43	0.08	0.17	0.14	0.05	0.05	0.66
FuzzCoder	CodeLlama	7B	0.17	0.13	0.83	0.71	0.71	0.81	1.82	0.03	0.11	0.35	0.08	0.26	0.50
FuzzCoder	DeepSeek-Coder	7B	2.19	1.83	1.01	1.88	1.25	0.97	2.40	1.87	3.42	2.96	1.66	0.44	**1.82**
FuzzCoder	CodeQwen	7B	1.83	0.54	1.27	1.39	1.37	1.32	2.98	0.97	2.41	1.12	2.69	2.43	1.69
FuzzCoder	CodeShell	7B	1.91	0.23	0.83	1.01	0.91	0.24	0.95	1.34	0.85	0.23	1.34	1.23	0.92
							LINT_XML								
AFL (Original)	-	-	11.21	1.75	1.49	0.13	3.37	5.42	0.82	0.11	0.00	1.13	0.24	0.08	2.15
AFL (LSTM)	-	-	2.82	2.06	4.60	0.00	3.09	0.00	3.01	0.00	0.00	4.64	3.24	0.00	1.96
AFL (Transformer)	-	-	5.71	2.90	3.01	0.00	2.99	3.08	2.82	0.00	0.00	7.15	0.00	0.00	2.31
FuzzCoder	StarCoder-2	7B	0.05	0.25	0.43	3.42	1.02	3.42	0.55	0.73	0.01	0.53	2.41	1.31	1.18
FuzzCoder	StarCoder-2	15B	0.13	0.13	0.54	2.72	1.73	2.43	0.48	0.54	0.34	0.71	3.42	2.33	1.29
FuzzCoder	CodeLlama	7B	0.31	0.32	1.31	12.31	2.43	1.27	0.83	0.34	0.45	0.65	2.45	1.43	2.01
FuzzCoder	DeepSeek-Coder	7B	0.99	0.00	0.49	14.28	8.31	0.36	0.84	0.72	0.41	2.61	1.42	9.80	**3.35**
FuzzCoder	CodeQwen	7B	0.68	0.82	0.19	19.51	6.42	0.00	1.65	0.91	0.28	3.63	0.41	2.51	3.08
FuzzCoder	CodeShell	7B	0.13	0.15	0.08	5.41	4.65	2.43	0.94	0.45	0.34	0.12	0.71	3.41	1.57
							MP3_GAIN								
AFL (Original)	-	-	0.65	0.22	0.15	0.09	0.91	0.40	0.08	0.09	0.01	0.23	0.28	0.17	0.27
AFL (LSTM)	-	-	1.60	1.68	1.19	0.33	0.65	0.00	1.95	1.61	0.00	1.16	3.46	3.44	1.42
AFL (Transformer)	-	-	2.70	1.01	0.93	0.00	0.52	0.19	1.25	0.17	0.00	1.02	3.20	3.87	1.24
FuzzCoder	StarCoder-2	7B	0.85	0.78	0.45	2.10	0.02	0.03	5.67	0.01	0.01	0.95	3.25	4.00	1.51
FuzzCoder	StarCoder-2	15B	0.90	0.82	0.50	2.20	0.03	0.04	5.80	0.01	0.01	1.00	3.30	4.10	1.56
FuzzCoder	CodeLlama	7B	0.80	0.76	0.40	2.00	0.01	0.02	5.50	0.00	0.01	0.90	3.20	3.90	1.46
FuzzCoder	DeepSeek-Coder	7B	0.76	0.75	0.36	2.13	0.00	0.00	6.44	0.00	0.00	1.25	3.30	4.12	**1.59**
FuzzCoder	CodeQwen	7B	1.09	0.83	0.48	0.82	1.05	0.00	2.72	0.00	0.00	1.72	3.21	3.50	1.29
FuzzCoder	CodeShell	7B	0.88	0.79	0.42	2.05	0.01	0.02	5.60	0.00	0.01	1.05	3.22	3.95	1.50
							IMAGE_MAGICK								
AFL (Original)	-	-	1.95	0.30	0.36	1.89	1.14	2.26	0.74	0.00	0.09	0.94	0.16	0.09	0.83
AFL (LSTM)	-	-	3.12	1.29	0.26	0.00	0.00	0.00	5.66	0.00	0.00	0.00	0.00	13.39	1.98
AFL (Transformer)	-	-	3.88	1.05	0.62	3.02	1.67	1.22	12.28	0.00	0.00	2.34	1.16	0.00	2.27
FuzzCoder	StarCoder-2	7B	2.05	1.82	0.70	1.40	0.00	0.80	8.90	1.30	0.00	3.20	8.10	3.15	2.62
FuzzCoder	StarCoder-2	15B	2.25	2.00	0.75	1.50	0.00	0.85	9.05	1.40	0.00	3.30	8.20	3.25	2.71
FuzzCoder	CodeLlama	7B	2.10	1.85	0.71	1.42	0.00	0.82	8.92	1.32	0.00	3.22	8.12	3.17	2.64
FuzzCoder	DeepSeek-Coder	7B	2.15	1.88	0.72	1.43	0.00	0.81	8.95	1.34	0.00	3.24	8.15	3.19	2.65
FuzzCoder	CodeQwen	7B	3.16	0.60	0.52	2.37	0.00	10.33	15.34	0.00	0.00	2.11	6.09	9.88	**4.20**
FuzzCoder	CodeShell	7B	2.12	1.86	0.73	1.44	0.00	0.83	8.97	1.35	0.00	3.25	8.16	3.20	2.66
							SPLIT_TIFF								
AFL (Original)	-	-	0.80	0.28	0.05	0.03	0.00	2.25	0.29	0.05	0.01	0.04	0.10	0.08	0.33
AFL (LSTM)	-	-	0.00	0.00	0.00	0.00	0.00	0.18	0.00	0.00	0.00	0.00	0.30	0.18	0.05
AFL (Transformer)	-	-	0.06	0.02	0.01	0.26	0.00	0.00	0.36	0.14	0.00	0.01	0.25	0.73	0.15
FuzzCoder	StarCoder-2	7B	0.15	0.05	0.10	2.10	0.00	0.70	0.05	0.00	0.00	0.01	0.02	0.01	0.27
FuzzCoder	StarCoder-2	15B	0.20	0.08	0.18	2.20	0.00	0.75	0.06	0.00	0.00	0.02	0.03	0.02	0.29
FuzzCoder	CodeLlama	7B	0.18	0.09	0.15	2.15	0.00	0.73	0.07	0.00	0.00	0.03	0.01	0.03	0.29
FuzzCoder	DeepSeek-Coder	7B	0.34	1.01	0.22	2.33	0.43	0.76	0.04	1.08	0.44	0.54	0.64	0.34	**0.68**
FuzzCoder	CodeQwen	7B	0.23	0.10	0.00	0.00	0.00	0.00	0.19	0.00	0.00	0.00	0.26	0.19	0.08
FuzzCoder	CodeShell	7B	0.14	0.07	0.11	2.12	0.00	0.72	0.03	0.00	0.00	0.01	0.02	0.01	0.27
							TRAN_JPEG								
AFL (Original)	-	-	1.41	0.35	0.15	0.27	0.41	1.18	0.18	0.08	0.01	0.32	0.21	0.11	0.39
AFL (LSTM)	-	-	2.68	0.98	0.52	0.82	0.00	0.00	5.80	0.94	0.00	1.44	3.67	2.15	1.58
AFL (Transformer)	-	-	0.14	1.11	0.66	1.32	1.30	1.94	2.42	1.96	0.00	1.83	2.82	2.76	1.52
FuzzCoder	StarCoder-2	7B	0.40	0.22	0.60	0.10	0.00	0.05	2.60	0.00	0.00	0.05	0.55	2.50	0.59
FuzzCoder	StarCoder-2	15B	0.50	0.28	0.65	0.15	0.00	0.08	2.70	0.01	0.00	0.10	0.60	2.60	0.64
FuzzCoder	CodeLlama	7B	0.45	0.25	0.58	0.12	0.00	0.07	2.55	0.00	0.00	0.07	0.54	2.45	0.59
FuzzCoder	DeepSeek-Coder	7B	0.36	0.21	0.56	0.00	0.00	0.00	2.52	0.00	0.00	0.00	0.53	2.40	0.55
FuzzCoder	CodeQwen	7B	3.40	0.54	0.86	0.45	0.53	0.54	1.29	1.13	0.54	2.11	6.21	1.34	**1.58**
FuzzCoder	CodeShell	7B	0.42	0.23	0.54	0.08	0.00	0.06	2.50	0.00	0.00	0.03	0.52	2.35	0.56

Table 2: Evaluation results (EPM, ‰) of multiple models. Bitflip a/b denotes $a * b$ bits are flipped as a whole. Arith a/b denotes the $a * b$ bits for addition and subtraction operations.

OBJDUMP_ELF, LINT_XML, MP3GAIN_MP3, IMAGEMAGICK_GIF, SPLIT_TIFF, and TRAN_JPEG. In this section, we provide the details, results, and analysis of the experiments.

5.1 Implementation Details

By performing fuzzy tests using AFL[6], we collect the original and variant inputs of successful attacks as a training set (nearly 30K SFT pairs). Our

[6] https://lcamtuf.coredump.cx/afl/

Method	Base	Size	READ_ELF	OBJ_DUMP	NM	LINT_XML	MP3_GAIN	IMAGE_MAGICK	SPLIT_TIFF	TRAN_JPEG	Avg.
AFL (Original)	-	-	0	0	0	117	68	0	95	0	35
AFL (LSTM)	-	-	0	0	0	55	53	0	42	0	19
AFL (Transformer)	-	-	0	0	0	61	45	0	77	0	23
FuzzCoder	StarCoder-2	7B	2	3	1	100	150	12	110	1	47
FuzzCoder	StarCoder-2	15B	**4**	5	2	120	180	**15**	130	2	57
FuzzCoder	CodeLlama	7B	3	2	0	90	140	10	100	1	43
FuzzCoder	DeepSeek-Coder	7B	2	4	0	**130**	**230**	3	**224**	**3**	75
FuzzCoder	CodeQwen	7B	1	**9**	0	114	209	4	221	2	**70**
FuzzCoder	CodeShell	7B	3	6	1	95	160	11	105	1	48

Table 3: Number of crashes of different models on eight datasets.

	READ_ELF				OBJ_DUMP				NM				LINT_XML			
	Line	Branch	Function	Avg.	Line	Branch	Function	Avg.	Line	Branch	Function	Avg.	Line	Branch	Function	Avg.
AFL (Original)	7.9	7.3	9.9	8.4	1.7	1.1	2.8	1.9	0.3	0.1	1.1	0.5	8.2	8.0	11.0	9.1
AFL (LSTM)	7.3	6.6	9.0	7.6	1.6	1.0	2.8	1.8	0.3	0.2	1.1	0.5	8.1	7.8	10.9	8.9
AFL (Transformer)	6.6	5.9	8.2	6.9	1.6	1.0	2.7	1.8	0.3	0.1	1.0	0.5	8.0	7.7	11.0	8.9
FuzzCoder (Deepseek-Coder)	14.9	16.5	15.4	15.6	2.0	1.5	3.1	2.2	0.6	0.3	1.9	0.9	9.2	9.4	11.8	10.1
FuzzCoder (CodeQwen)	14.5	15.9	15.2	15.2	2.0	1.5	3.1	2.2	0.6	0.4	1.9	1.0	8.7	8.8	11.3	9.6
	MP3_GAIN				IMAGE_MAGICK				SPLIT_TIFF				TRAN_JPEG			
AFL (Original)	53.5	41.3	58.1	51.0	87.5	50.0	100.0	79.2	1.0	1.4	1.4	1.3	17.8	22.6	27.5	22.6
AFL (LSTM)	53.2	40.8	58.1	50.7	87.5	50.0	100.0	79.2	0.9	1.3	1.1	1.1	15.5	18.8	26.3	20.2
AFL (Transformer)	54.0	41.5	58.1	51.2	87.5	50.0	100.0	79.2	1.0	1.4	1.4	1.3	15.4	18.3	26.3	20.0
FuzzCoder (Deepseek-Coder)	54.9	43.2	59.1	52.4	87.5	50.0	100.0	79.2	1.0	1.6	1.4	1.3	19.0	24.7	27.9	23.9
FuzzCoder (CodeQwen)	54.9	42.8	59.1	52.3	87.5	50.0	100.0	79.2	1.0	1.6	1.4	1.3	18.2	23.1	27.2	22.8

Table 4: Coverate rate (%) of different models on 8 datasets.

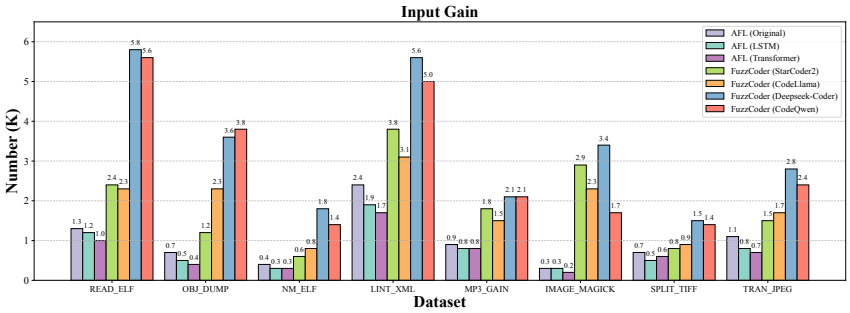

Figure 4: Comparison between the baselines and FuzzCoder.

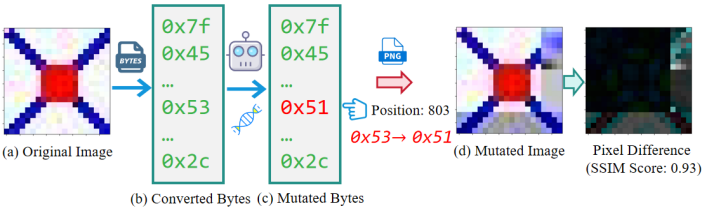

Figure 5: Comparison between the original JPG file and the JPG file after blur test

model based on open-source code LLMs CodeLlama, Deepseek-Coder, and CodeQwen is trained for 3 epochs with a cosine scheduler, starting at a learning rate of 5e-5 (3% warmup steps). We use the AdamW (Loshchilov and Hutter, 2017) optimizer with a batch size of 1024 (max length 4K).

5.2 Methods

AFL (Original): The original AFL with the heuristic mutation rules is used as a baseline. **AFL (LSTM):** We use the encoder-decoder-based LSTM network without pre-training to decide the mutation position and strategy. **AFL (Transformer):** The encoder-decoder-based Transformer without pre-training is incorporated into the AFL tool to improve the effectiveness of the fuzzing test. **StarCoder-2:** StarCoder-2 models with 3B, 7B, and 15B parameters are trained on 3.3 to 4.3 trillion tokens, supporting hundreds of programming languages. **Code-Llama:** Code-Llama is a family of code large language models based on Llama 2, providing infilling and long context capabilities. **DeepSeek-Coder:** Deepseek-coder is a series of open-source code models with sizes from 1.3B to 33B, pre-trained from scratch on 2 trillion tokens. **CodeQwen:** CodeQwen with 7B parameters supports 92 languages and 64K tokens.

5.3 Evaluation Metrics

Effective proportion of mutation (EPM): For each mutation of the seed sample in the queue, a mutation location is selected, and then the corresponding mutation strategy is carried out for a mutation location. The effective proportion of mutations (‰) can be used to evaluate the effectiveness of different methods.

Number of Crashes (NC): This indicator refers to the number of input samples that cause the program to crash during fuzz testing and is used to measure the number of malicious inputs and the number of vulnerabilities.

5.4 Main Results

Results of EPM In Table 2, we find that the FUZZCODER generally has better EPM than the AFL (Original) in each of the 8 programs and different LLMs have their own advantages in different programs. The results demonstrate that the code LLMs with the powerful understanding and generation capabilities can further bring improvement for the fuzzing test, compared to the AFL with small models.

Results of NC In Table 3, our vulnerability findings for `READ_ELF` and `NM` programs have 0 results on AFL (Original), AFL (LSTM and Transformer), which indicates that these two datasets are hard to vulnerabilities in the limited time. It shows that the mutation sequences from the LLMs easily lead to the crash for the program to be tested.

6 Discussions and Analysis

Input Gain (IG) Figure 4 shows the number of new paths of changes in the execution of code blocks found during fuzz testing of the target program. We can observe that FUZZCODER significantly improves the performance compared to the heuristic methods.

Coverage Rate In Table 4, we report the coverage rate of different models, including line coverage, branch coverage, and function coverage. Line coverage refers to the ratio of whether each line of code has been executed at the time of the program under test fuzzing, and branch coverage refers to the ratio of whether each conditional branch has been executed at the time of the program under test fuzzing. By looking at these two metrics, we can know whether the test cases mutated by the Fuzzer can trigger more complete paths more effectively, so the higher these two metrics, the better.

Case study In Figure 5, we take the `JPEG_TRANS` program as an example. In Figure 5, the original Image will get Mutated Image after several rounds of fuzzing test. We use the big language model to guide the mutation of Image. For example, where Original Image was 0x53, it becomes 0x51. And the SSIM Score of Mutated Image vs. Original Image is 0.93. The Mutated Image is then fed into the JPEGTRAN program, which triggers a new code path or a program crash.

7 Related Work

Fuzzing Test Inspired by the success of sequence-to-sequence learning (s2s) in many NLP tasks (Vaswani et al., 2017; Yang et al., 2020, 2022b,a), the fuzzing test approaches use s2s to train neural networks to learn generative models of the input formats for fuzzing. For different input formats and the target program, random mutation of the inputs makes it hard to find the vulnerable positions to fuzz the program. Deep-learning-based methods (Godefroid et al., 2017; He et al., 2019; Patra and Pradel, 2016; Yang et al., 2024) present a technique to use LSTMs to learn grammar for PDF objects using a character-level model, which can then be sampled to generate new inputs. Instead of learning grammar, our technique uses neural networks to learn a function to predict promising locations in a seed file to perform mutations. The previous methods are hindered by a small number of parameters and the training corpora lack common knowledge of the byte sequence, codes, and reasoning. Recently, researchers (Xia et al., 2024; Deng et al., 2023) directly leverage prompt engineering to inspire the instruct-following capability of LLMs for effective fuzzing.

Domain-specific Large Language Model Large language models (LLMs) (Touvron et al., 2023a,b; Achiam et al., 2023; Bai et al., 2023) based on the decoder-only Transformer architecture have become a cornerstone in the realm of natural language processing (NLP). The pre-training on a vast corpus of internet text, encompassing billions of tokens enables LLMs to understand and generate human-style responses, making them highly versatile as zero-short learners. Further, code LLMs tailored for software engineering tasks push boundaries of

code understanding and generation (Chai et al., 2024; Guo et al., 2023, 2024b; Rozière et al., 2023; Guo et al., 2024a; Wu et al., 2024; Slagle, 2024). The code LLM supports many code-related works, such as code translation, code generation, code refinement, program repair, and fuzzing. Recent methods tailored for fuzzing (Xia et al., 2024; Yao et al., 2024; Deng et al., 2023) relying on common LLMs without domain-specific instruction tuning can not effectively unleash the potential of LLMs in the field of fuzzing.

8 Conclusions

In this paper, we present FUZZCODER, a series of fine-tuned large language models for the fuzzing test. First, we collect the Fuzz-Instruct dataset based on a self-instruct strategy, which contains multiple programs to improve the generalization ability of LLMs on fuzzing operations. Then, to easily evaluate the performance of existing LLMs on fuzzing test, we also introduce the Fuzz-Bench evaluation benchmark dataset with eight programs. Besides, we also introduce the mixture-of-adapter strategy to further enhance the instruction tuning performance. Moreover, extensive experimental results on our FUZZCODER demonstrate the effectiveness of our FUZZCODER for fuzzing test.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Linzheng Chai, Jian Yang, Tao Sun, Hongcheng Guo, Jiaheng Liu, Bing Wang, Xiannian Liang, Jiaqi Bai, Tongliang Li, Qiyao Peng, et al. 2024. xcot: Cross-lingual instruction tuning for cross-lingual chain-of-thought reasoning. *arXiv preprint arXiv:2401.07037*.

Chris Cummins, Pavlos Petoumenos, Alastair Murray, and Hugh Leather. 2018. Compiler fuzzing through deep learning. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 95–105.

Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, pages 423–435.

Patrice Godefroid, Hila Peleg, and Rishabh Singh. 2017. Learn&fuzz: Machine learning for input fuzzing. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 50–59. IEEE.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. http://www.deeplearningbook.org.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. 2024a. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.

Hongcheng Guo, Jian Yang, Jiaheng Liu, Liqun Yang, Linzheng Chai, Jiaqi Bai, Junran Peng, Xiaorong Hu, Chao Chen, Dongfeng Zhang, et al. 2023. Owl: A large language model for it operations. *arXiv preprint arXiv:2309.09298*.

Hongcheng Guo, Wei Zhang, Anjie Le, Jian Yang, Jiaheng Liu, Zhoujun Li, Tieqiao Zheng, Shi Xu, Runqiang Zang, Liangfan Zheng, et al. 2024b. Lemur: Log parsing with entropy sampling and chain-of-thought merging. *arXiv preprint arXiv:2402.18205*.

Jianmin Guo, Yu Jiang, Yue Zhao, Quan Chen, and Jiaguang Sun. 2018. Dlfuzz: Differential fuzzing testing of deep learning systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 739–743.

Jingxuan He, Mislav Balunović, Nodar Ambroladze, Petar Tsankov, and Martin Vechev. 2019. Learning to fuzz from symbolic execution with application to smart contracts. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 531–548.

Linghan Huang, Peizhou Zhao, Huaming Chen, and Lei Ma. 2024. Large language models based fuzzing techniques: A survey. *arXiv preprint arXiv:2402.00350*.

Jun Li, Bodong Zhao, and Chao Zhang. 2018. Fuzzing: a survey. *Cybersecurity*, 1(1):1–13.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

Valentin JM Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J Schwartz, and Maverick Woo. 2019. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering*, 47(11):2312–2331.

Jibesh Patra and Michael Pradel. 2016. Learning to fuzz: Application-independent fuzz testing with probabilistic, generative models of input data. *TU Darmstadt, Department of Computer Science, Tech. Rep. TUD-CS-2016-14664*.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Kevin Slagle. 2024. Spacebyte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NIPS 2017*, pages 5998–6008.

Changhan Wang, Kyunghyun Cho, and Jiatao Gu. 2020. Neural machine translation with byte-level subwords. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 9154–9160. AAAI Press.

Anjiang Wei, Yinlin Deng, Chenyuan Yang, and Lingming Zhang. 2022. Free lunch for testing: Fuzzing deep-learning libraries from open source. In *Proceedings of the 44th International Conference on Software Engineering*, pages 995–1007.

Shangda Wu, Xu Tan, Zili Wang, Rui Wang, Xiaobing Li, and Maosong Sun. 2024. Beyond language models: Byte models are digital world simulators. *arXiv preprint arXiv:2402.19155*.

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. *arXiv preprint arXiv:2308.04748*.

Danning Xie, Yitong Li, Mijung Kim, Hung Viet Pham, Lin Tan, Xiangyu Zhang, and Michael W Godfrey. 2022. Docter: documentation-guided fuzzing for testing deep learning api functions. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 176–188.

Jian Yang, Shuming Ma, Dongdong Zhang, Shuangzhi Wu, Zhoujun Li, and Ming Zhou. 2020. Alternating language modeling for cross-lingual pre-training. In *AAAI 2020*, pages 9386–9393.

Jian Yang, Yuwei Yin, Shuming Ma, Dongdong Zhang, Zhoujun Li, and Furu Wei. 2022a. High-resource language-specific training for multilingual neural machine translation. In *IJCAI 2022*, pages 4461–4467.

Jian Yang, Yuwei Yin, Shuming Ma, Dongdong Zhang, Shuangzhi Wu, Hongcheng Guo, Zhoujun Li, and Furu Wei. 2022b. UM4: unified multilingual multiple teacher-student model for zero-resource neural machine translation. In *IJCAI 2022*, pages 4454–4460.

Liqun Yang, Chaoren Wei, Jian Yang, Jinxin Ma, Hongcheng Guo, Long Cheng, and Zhoujun Li. 2024. Seq2seq-afl: Fuzzing via sequence-to-sequence model. *International Journal of Machine Learning and Cybernetics*, pages 1–19.

Dongyu Yao, Jianshu Zhang, Ian G Harris, and Marcel Carlsson. 2024. Fuzzllm: A novel and universal fuzzing framework for proactively discovering jailbreak vulnerabilities in large language models. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4485–4489. IEEE.

Quanjun Zhang, Tongke Zhang, Juan Zhai, Chunrong Fang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. A critical review of large language model on software engineering: An example from chatgpt and automated program repair. *arXiv preprint arXiv:2310.08879*.